Звіт до лабораторної 6

«Atomic Swap»

студента ММШІ-1 Красікова Антона

Завдання:

Реалізувати Atomic swap для тристороннього обміну умовною криптовалютою.

Вимоги:

- Кожна сторона має бути представлена окремим процесом.
- Необхідно використати принцип роботи смарт-контракту HTLC (Hashed Timelock Contract).
- Забезпечити логування роботи системи.

1. Спосіб розв’язання

Для реалізації завдання було обрано мову програмування Python. У контексті криптовалютних мереж ізоляція пам'яті є критичною, тому вузли мережі повинні працювати як незалежні процеси, а не потоки, що ділять спільну пам'ять.

Для симуляції розподіленої мережі було створено програмний вузол (Node), який запускається у трьох незалежних екземплярах (процесах ОС) на різних мережевих портах, імітуючи трьох учасників: Алісу, Боба та Чарлі. Взаємодія між вузлами відбувається через REST API (HTTP-запити).

Основу безпечного обміну без довіри (trustless) складає механізм **HTLC**, який використовує:

1. **Hashlock:** Блокування коштів криптографічним хешем (SHA-256) від секретного ключа. Кошти можна розблокувати лише пред'явивши оригінальний секрет.
2. **Timelock:** Механізм таймауту. Якщо секрет не розкрито за відведений час, кошти автоматично повертаються відправнику.

Алгоритм тристороннього обміну:

Згідно зі стандартами реалізації Atomic Swap, учасники формують ланцюжок контрактів із спадними таймаутами (щоб унеможливити крадіжку коштів на останніх етапах).

1. Аліса генерує секрет, створює хеш та блокує кошти для Чарлі (таймаут: T3).

2. Чарлі бачить хеш і блокує кошти для Боба з тим самим хешем (таймаут: T2 < T3).
3. Боб бачить хеш і блокує кошти для Аліси (таймаут: T1 < T2).
4. Аліса розкриває секрет вузлу Боба, щоб забрати кошти. Боб дізнається секрет і забирає кошти у Чарлі. Чарлі дізнається секрет і забирає кошти в Аліси.

2. Деталі реалізації

Систему побудовано з використанням фреймворку FastAPI, що є індустріальним стандартом для побудови мікросервісів на Python.

Основні техніки та компоненти:

- **Моделювання стану:** Кожен вузол має локальний стан (баланс, статус контракту, заблокована сума). Це імітує роботу локального леджера.
- **Типізація та валідація даних:** Використано бібліотеку Pydantic для валідації вхідних даних (моделі LockRequest, ClaimRequest). У сфері смарт-контрактів валідація є критичним елементом безпеки.
- **Криптографія:** Використано стандартний модуль hashlib для генерації SHA-256 хешів.
- **Логування:** Для логування використано модуль logging із зазначенням міток часу, рівня важливості (INFO, WARNING, ERROR) та імені процесу. Це дозволяє здійснювати повноцінний аудит роботи вузла.
- **API ендпоінти:**

- POST /lock - створення смарт-контракту, списання коштів з балансу та встановлення умов (hashlock, timeout).
- POST /claim - виконання контракту, перевірка криптографічного доказу та передача коштів.
- POST /refund - механізм захисту: повернення коштів у разі закінчення таймауту та відсутності доказу.

3. Результати

У ході тестування було успішно розгорнуто три незалежні вузли (порти 5001, 5002, 5003).

Крок 1. Ініціалізація та блокування коштів

Було згенеровано секрет `my_secret` із хешем 1f9233a.... Сторони послідовно заблокували кошти через API-запити до /lock зі спадними таймаутами (300c, 200c, 100c).

Крок 2. Обмін та розкриття секрету

Аліса ініціювала виконання контракту на вузлі Боба. Система успішно верифікувала секрет, після чого спрацював ефект доміно: кожна наступна сторона використовувала розкритий секрет для отримання коштів.

Нижче наведено логи вузлів Аліси, Боба та Чарлі, які демонструють успішну валідацію криптографічного доказу та переказ коштів:

```
(distributed_env) C:\Users\chess\uni\distributed_systems\labs\lab6>python node.py 5001 Alice
2026-04-05 22:53:38,856 - Alice - INFO - FUNDS LOCKED! 10 coins reserved for 'Charlie'. Hashlock: 1f9233a121057dcedc0b8e
6d32fb605d2137879e3974ee6570a07062975845d9
2026-04-05 22:54:45,996 - Alice - WARNING - CRITICAL: Contract claimed! Secret revealed: 'my_secret'
2026-04-05 22:54:45,996 - Alice - INFO - 10 coins successfully transferred to Charlie.
```

```
(distributed_env) C:\Users\chess\uni\distributed_systems\labs\lab6>python node.py 5002 Bob
2026-04-05 22:54:04,548 - Bob - INFO - FUNDS LOCKED! 10 coins reserved for 'Alice'. Hashlock: 1f9233a121057dcedc0b8e6d32
fb605d2137879e3974ee6570a07062975845d9
2026-04-05 22:54:22,119 - Bob - WARNING - CRITICAL: Contract claimed! Secret revealed: 'my_secret'
2026-04-05 22:54:22,119 - Bob - INFO - 10 coins successfully transferred to Alice.
```

```
(distributed_env) C:\Users\chess\uni\distributed_systems\labs\lab6>python node.py 5003 Charlie
2026-04-05 22:53:53,973 - Charlie - INFO - FUNDS LOCKED! 10 coins reserved for 'Bob'. Hashlock: 1f9233a121057dcedc0b8e6d
32fb605d2137879e3974ee6570a07062975845d9
2026-04-05 22:54:36,104 - Charlie - WARNING - CRITICAL: Contract claimed! Secret revealed: 'my_secret'
2026-04-05 22:54:36,104 - Charlie - INFO - 10 coins successfully transferred to Bob.
```

Висновок:

Тристоронній Atomic Swap виконано успішно. Всі процеси залишилися ізольованими. Механізм HTLC довів свою працездатність: обмін відбувся атомарно і без залучення довіреної третьої сторони. Умови безпеки (спадні таймаути та хеш-замки) дотримані.

Код системи (папка lab6):

`https://github.com/chesswondo/distributed-systems-labs`